

Classification - Logistic Regression

Chapter 3: Classification

Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, Gurelien Geron, 3e, O'Reilly Media, 2022.

Classification

- Classification refers to classifying data into discrete outcomes.
- Types of classification:
 - Binary classification:** Only two classes are there, 0 and 1; $y \in \{0,1\}$.
 - Multiclass classification:** More than two classes are possible, 1, 2, 3, etc.; $y \in \{1,2,3, \dots, p\}$, where p is the number of classes.

Binary Classification Problem

- In binary classification problems, the hypothesis y can take on only two values:
 - 0 or negative (-)
 - 1 or positive (+)
- Hence $y \in \{0,1\}$.

The Sigmoid Function

- The Sigmoid function σ is given as: $\sigma(z) = \frac{1}{1+e^{-z}}$, where e is the Euler's constant with value 2.71828.

- It asymptotes at 1 and 0.

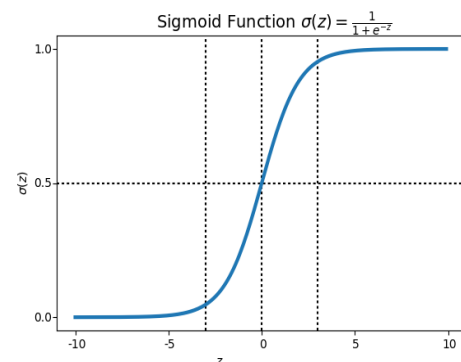
$$z = 0, e^0 = 1 \text{ hence } \sigma(z) = 0.5$$

$$z > 0 (+ve), e^{-z} < 1, \text{ hence } \sigma(z) > 0.5$$

$$z \rightarrow \infty, e^{-\infty} \rightarrow 0, \text{ hence } \sigma(z) = 1$$

$$z < 0 (-ve), e^{+z} > 1, \text{ hence } \sigma(z) < 0.5$$

$$z \rightarrow -\infty, e^{\infty} \rightarrow \infty, \text{ hence } \sigma(z) = 0$$



- The score z is often called the *logit*.
- This name comes from the fact that the logit function, defined as $\text{logit}(p) = \log(p/(1-p))$, is the inverse of the logistic function.

Logistic Regression for Binary Classification

- Also known as **Logit Regression**, **Maximum Entropy Classification (MaxEnt)**, or the **Log-linear Classifier**.
- It is a probability-based predictive algorithm used for binary classification tasks.
- The model outputs a probability between 0 and 1 using the sigmoid (logistic) function, $0 \leq \hat{y} \leq 1$.

Hypothesis Formulation

- Linear regression hypothesis: $h_{\theta}(x) = \hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_m x_m$
- To constrain output between 0 and 1, logistic regression uses the sigmoid function:

$$\hat{y} = \sigma(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_m x_m), \text{ where } \sigma \text{ is the Sigmoid function.}$$

$$\hat{y} = \frac{1}{1 + e^{-(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_m x_m)}}$$

The sigmoid function maps any real-valued input to a probability-like output, ideal for binary classification.

Decision Rule

- Logit, $z = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_m x_m$

- If $z > 0$, predict $\hat{y} = 1$
- If $z < 0$, predict $\hat{y} = 0$
- If $z = 0$, it may be assigned randomly, biased toward the majority class, or removed from training.
- z is also called the decision score.

Probability Interpretation

- Note that $\hat{y}$ will give the probability that the output is 1.
- For example, $\hat{y} = 0.7$ gives a probability of 70% that our output is 1.
- The probability that the prediction is 0 is just the complement of the probability that it is 1.
- $\hat{y} = P(y = 1|x; \theta) = 1 - P(y = 0|x; \theta)$
- $P(y = 1|x; \theta) + P(y = 0|x; \theta) = 1$

Cost Function

Summarizing the model

- For a training set having n samples and m features:
Training set $\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}) \dots, (x^{(n)}, y^{(n)})\}$

Feature vector $x \in \begin{bmatrix} x_0 \\ x_1 \\ \dots \\ x_m \end{bmatrix}$, where $x_0 = 1$

Output label $y \in \{0, 1\}$

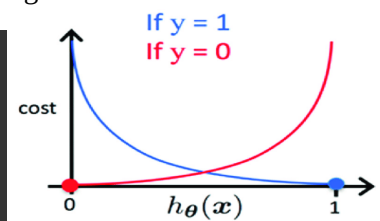
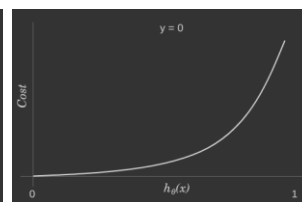
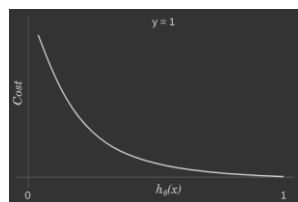
Hypothesis $h_{\theta}(x) = \hat{y} = \frac{1}{1 + e^{-\theta^T x}}$

- Let us use the same cost function as used for Linear regression:

$$\begin{aligned} \text{Cost function } J(\theta) &= \frac{1}{n} \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)})^2 \\ &= \frac{1}{n} \sum_{i=1}^n \left(\frac{1}{1 + e^{-\theta^T x^{(i)}}} - y^{(i)} \right)^2 \end{aligned}$$

- Plotting this cost function gives a wavy, non-convex plot with many local minima.
- The objective of training is to set the parameter vector θ so that the model estimates high probabilities for positive instances ($y = 1$) and low probabilities for negative instances ($y = 0$).
- This idea is captured by the cost functions shown in following equations for a single training instance x .

- For $y = 1$:
 $\text{Cost}(\hat{y}^{(i)}, y^{(i)}) = -\log(\hat{y}^{(i)})$
- For $y = 0$:
 $\text{Cost}(\hat{y}^{(i)}, y^{(i)}) = -\log(1 - \hat{y}^{(i)})$



- If the correct answer y is 1, then the cost function will be 0 if the hypothesis function outputs 1. If the hypothesis approaches 0, then the cost function will approach infinity.
- If the correct answer y is 0, then the cost function will be 0 if the hypothesis function also outputs 0. If the hypothesis approaches 1, then the cost function will approach infinity.
- Combining into one cost function:

$$J(\theta) = -\frac{1}{n} \sum_{i=1}^n [y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$$

- The cost function over the whole training set is simply the average cost over all training instances.
- This cost function guarantees that $J(\theta)$ is convex for logistic regression.
- Rewriting as follows:

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n \text{Cost}(\hat{y}^{(i)}, y^{(i)})$$

- This cost function is also called **Log Loss function** or **Cross Entropy function**.

Gradient Descent

- Recalling general form of gradient descent, to minimize $J(\theta)$:
Repeat until converge:
$$\theta_j = \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta), \text{ simultaneously update all } \theta_j$$

or
$$\theta_j = \theta_j - \frac{\alpha}{n} \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)}) x_j^{(i)}$$
- The form is pretty similar to the one used in Linear regression, the difference is in the definition of $\hat{y}$.
- This simplification of the derivative uses derivative of Sigmoid function to bring it to this form which is similar to that of linear regression. So if the output function is different (let's say Softmax), then this derivative comes out to be different.
- Some faster alternates to gradient descent for optimization are conjugate gradient, BFGS (Broyden-Fletcher-Goldfarb-Shanno), and L-BFGS (Limited memory BFGS).

Decision Boundary

- The decision boundary separates the classes/labels in a feature space.
- It is created by the hypothesis function.
- It is called the Classifier, could be linear or polynomial.
- If number of features, $m=2$, the boundary equation defines a line,
- If $m=3$, the boundary equation defines a plane, and
- If $m > 3$, the boundary equation defines a hyperplane.

Multiclass Classification

- Multiclass classification is required when there are more than two categories.
- Also called Multinomial classification.
- Instead of $y \in \{0,1\}$, now $y \in \{1, \dots, p\}$
Total classes= p
- Sigmoid function based Logistic regression is inherently binary, therefore it cannot be used directly for multiclass classifications.

One-vs-all (OvA) / One-vs-rest Approach

- Since $y = \{1, \dots, p\}$, the problem is divided into p binary classification problems; in each one, the probability that y is a member of one of our classes is predicted.
- Total classifiers trained: p .
- Divide the problem into p classifiers.
- Train p binary classifiers $\hat{y}_1, \hat{y}_2, \dots, \hat{y}_p$ separately, applying logistic regression.
For each classifier choose one class and lump all the other classes into a single second class.
- To make a prediction on new x , pick the class that maximizes $\hat{y}$. new feature data point that belongs to no class

One-vs-one (OvO) Approach

- The one-vs-one approach splits the dataset into one dataset for each class versus every other class.
- See which class wins the most duels.
- Total classifiers trained: $p \times (p-1)/2$ $[(p-1)(p-2) + \dots + 0 = p \times (p-1)/2]$
- The main advantage of OvO is that each classifier only needs to be trained on the part of the training set for the two classes that it must distinguish. Some algorithms (such as Support Vector Machine classifiers) scale poorly with the size of the training set, so for these algorithms OvO is preferred since it is faster to train many classifiers on small training sets than training few classifiers on large training sets. For most binary classification algorithms, however, OvA is preferred.

benefit : low trainig data hence, faster

Multilabel and Multioutput Classification

- Binary classification means a classification problem where there are two classes / labels only.
- Multiclass classification means classification problem where the task is to classify between more than two classes.
- Multilabel classification means a classification problem where there are multiple binary labels as output.
- Multioutputmulticlass classification (or multioutput classification) is simply a generalization of multilabel classification where each label can be multiclass.

Classification Evaluation - The Confusion Matrix

- A **confusion matrix** is a key tool used to assess the performance of a classifier by summarizing correct and incorrect predictions broken down by class.
- It helps us understand not just how many errors were made, but also what types of errors occurred, giving deeper insight into model behavior.
- A binary classifier can have four classification results: true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN); the first two are correct classifications.

True Positive: TP is the number of true positives.

correction x predicted T/f

- The ML model predicted positive and it is true.
- **Example:** The ML model predicted that a man has cancer and he actually has it.

True Negative: TN is the number of true negatives.

- The ML model predicted negative and it is true.
- **Example:** The ML model predicted that a man does not have cancer and he actually does not have it.

False Positive: FP is the number of false positives.

- Also called type I error.
- The ML model predicted positive and it is false.
- **Example:** The ML model predicted that a man has cancer but he actually does not have it.

False Negative: FN is the number of false negatives.

- Also called type II error.
- The ML model predicted negative and it is false.
- **Example:** The ML model predicted that a man does not have cancer but he has it.

Constructing a Confusion Matrix

- Predict outputs for all instances in the test/validation set.
- Compare predictions with actual labels.
- Count TP, TN, FP, FN values.
- Arrange them in a matrix: Rows = Actual class, Columns = Predicted class
- A perfect classifier will have values only along the main diagonal.

	Predicted Negative	Predicted Positive	
Actual Negative	True negatives	False positives	Type I error
Actual Positive	False negatives	True positives	Type II error

Accurate results

The aim of classification is to reduce type I and II errors!

Evaluation Metrics Based on Confusion Matrix

Accuracy

- Classification accuracy is the ratio of correct predictions to total predictions made.
- $$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Number of total predictions}} = \frac{TP+TN}{TP+FP+FN+TN}$$
- Accuracy tells you how many times the ML model was correct overall.

Precision

- Precision is the ratio of correct positive predictions to all the predicted positives.
- $$\text{Precision} = \frac{\text{Number of correct positive predictions}}{\text{Number of total predicted positives}} = \frac{TP}{TP+FP}$$
- To increase precision, try to minimize type I error (FP).
- Precision is how good the model is at predicting a specific category.

	PN	PP
AN	TN	FP
AP	FN	TP

Recall Score

- Also called sensitivity or true positive rate (TPR).
- Recall score is the ratio of correct positive predictions to all the actual positives.
- $$\text{Recall Score} = \frac{\text{Number of correct positive predictions}}{\text{Number of total actual positives}} = \frac{TP}{TP+FN}$$
- To increase precision, try to minimize type II error (FN).
- Recall tells you how many times the model was able to detect a specific category.

	PN	PP
AN	TN	FP
AP	FN	TP

F1 score

- The F1 score is the harmonic mean (a kind of average used for ratios).
- Whereas the regular mean treats all values equally, the harmonic mean gives much more weight to low values.
- $$F1_{\text{score}} = \frac{2}{\frac{1}{\text{precision}} + \frac{1}{\text{recall}}} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$
- Used when both FP and FN are almost equally important.
- F1 score represents a balance between the recall and precision, where the relative contributions of both are equal.
- It gives good results only when both recall and precision are good; this is what harmonic mean does.

Which metric to use?

Skewness of Dataset

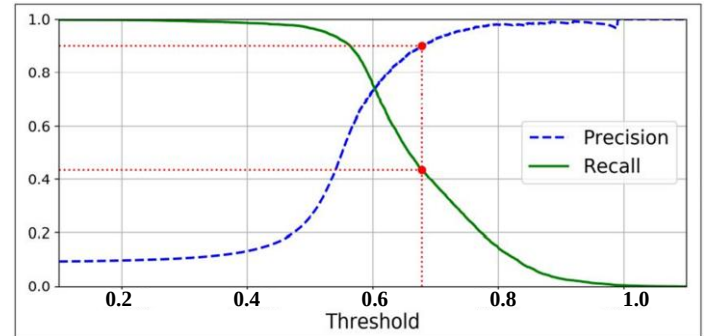
- In a skewed dataset, the smaller class will naturally be harder to detect due to the smaller number of samples.
- Accuracy works well for balanced classes. In the presence of imbalanced classes, accuracy suffers from a paradox where a model is highly accurate but lacks predictive power.
- Use precision and recall to focus on small positive class; the class under focus should be declared as the positive class.
- Use F1 score when both positive and negative class detection is equally important.

Use-case

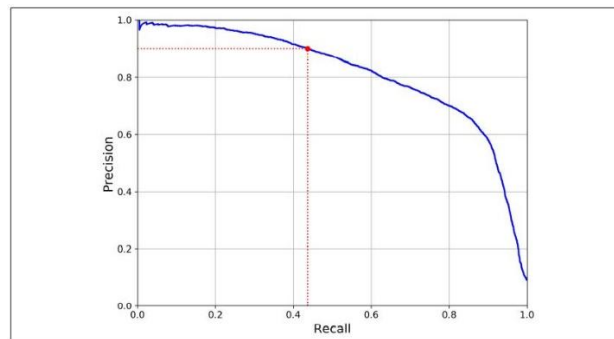
- See which type is more harmful or creating a greater impact: FP or FN.
 - If FP is more hurtful, use precision.
 - If FN is more hurtful, use recall.
- The F1 score favors classifiers that have similar precision and recall.

Decision Threshold and Precision/Recall Tradeoff

- In most applications increasing precision reduces recall, and vice versa. This is called the precision/recall tradeoff.
- Precision and recall scores can be adjusted by adjusting the decision threshold of the decision function.
- To select a good precision/recall trade off, plot precision and recall values for different decision thresholds and then choose the one that suits the application.



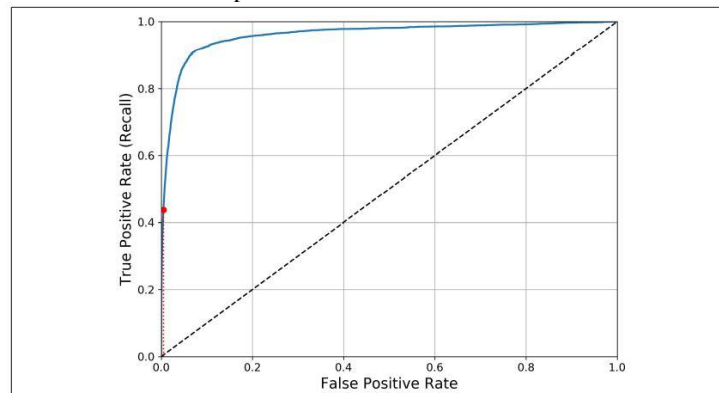
- Another way to select a good precision/recall tradeoff is to plot precision directly against recall, for different threshold values.



The ROC Curve

- The receiver operating characteristic (ROC) curve is another common tool used with binary classifiers.
- The method was originally developed for operators of military radar receivers starting in 1941, which led to its name.
- ROC curve is a performance measurement for the classification problems at various threshold settings.
- ROC is a probability curve and area under the curve (AUC) represents the degree or measure of separability.
- The ROC curve plots the true positive rate (TPR) against the false positive rate (FPR).
- The FPR, is the ratio of negative instances that are incorrectly classified as positive.
 - $$FPR = \frac{\text{Number of incorrect negative predictions}}{\text{Total actual negatives}} = \frac{FP}{FP+TN}$$
- To plot the ROC curve, you first need to compute the TPR and FPR for various threshold values:

$$tpr = tp/(TP+FN)$$



Confusion Matrix for Multiclass Classification

- Unlike binary classification, there are no positive or negative classes here.
- TP, TN, FP and FN are calculated individually for each class.
- For a class:
 - True positives: the diagonal value.
 - False positives: the column values, except the diagonal value.
 - False negatives: the row values, except the diagonal value.
 - True negatives: all the remaining values.

Performance Metrics for Multiclass Classification

- Performance metric can be calculated both ways:
 - individually for each class
 - one cumulative/global score for all classes.

Cumulative Precision, Recall and F1 Score

Micro Scores

- This score is calculated by considering the total TP, total FP and total FN of the model.
- When calculated globally through micro scores all metrics become (precision, recall, F1 score and even accuracy) equal.
- Best for: Imbalanced class distributions when overall performance matters.
- Treats every individual prediction equally, regardless of class. Skewed toward performance on the most frequent classes.

Macro Scores

- This score calculates metrics for each class individually and then takes unweighted mean of the measures.
- Best for: Balanced importance of all classes (even if they are rare).
- Gives equal weight to all classes, so poor performance on a minority class significantly lowers the macro score.

Weighted F1

- This metric calculates weighted-averaged of the individual scores.
- The weights for each class are the total number of samples of that class.
- Best for: Reflecting overall performance while taking class imbalance into account.
- Balances between micro and macro, it doesn't ignore rare classes but doesn't let them dominate either.